

Ход выполнения работы

1. Создать таблицу с ограничениями и заполнить ее данными из пункта 3 предыдущей темы. Таблица должна содержать поле id на основе последовательности и ограничение типов primary key, check и not null (пример, проверка что котировка не может быть ноль). Попытаются выполнить CRUD-операции, нарушающие созданные ограничения.

Была создана таблица, содержащая поле id на основе последовательности и ограничения типов PRIMARY KEY, CHECK и NOT NULL. Запрос создания этой таблицы представлен на скриншоте 1.1:

```
-- Создание таблицы с ограничениями
CREATE TABLE ticks (
    id SERIAL PRIMARY KEY,
    ticker VARCHAR(10) NOT NULL,
    per INTEGER NOT NULL CHECK (per >= 0),
    date DATE NOT NULL,
    time TIME NOT NULL,
    open NUMERIC(10,2) NOT NULL CHECK (open > 0),
    high NUMERIC(10,2) NOT NULL CHECK (high > 0),
    low NUMERIC(10,2) NOT NULL CHECK (low > 0),
    close NUMERIC(10,2) NOT NULL CHECK (close > 0),
    vol INTEGER NOT NULL CHECK (vol >= 0),
    CONSTRAINT high_ge_low CHECK (high >= low)
)
```

Скриншот 1.1. Создание таблицы

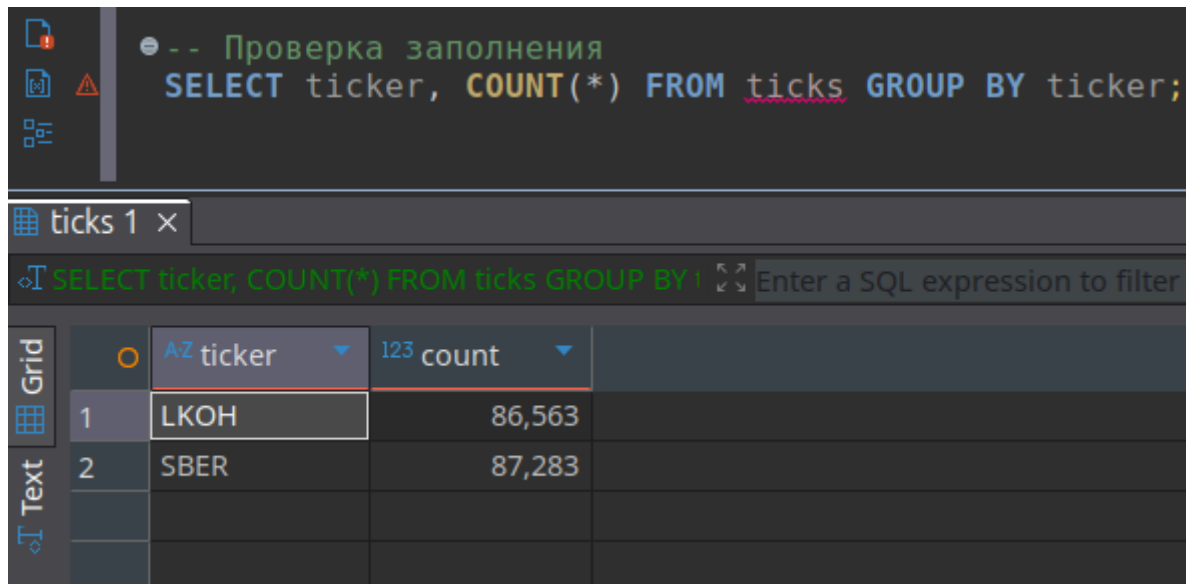
В созданную таблицу были вставлены данные из предыдущей практической работы. Запрос вставки в таблицу представлен на скриншоте 1.2:

```
-- Заполнение таблицы данными
COPY ticks (ticker, per, date, time, open, high, low, close, vol)
FROM '/home/staspy/Downloads/SBER_260101_260421.csv'
DELIMITER ';'
CSV HEADER;

COPY ticks (ticker, per, date, time, open, high, low, close, vol)
FROM '/home/staspy/Downloads/LKOH_260101_260421.csv'
DELIMITER ';'
CSV HEADER;
```

Скриншот 1.2. Вставка данных в таблицу

Была выполнена проверка, что данные были вставлены правильно. Запрос и результат запроса проверки на то, что все данные были успешно вставлены, представлены на скриншоте 1.3:

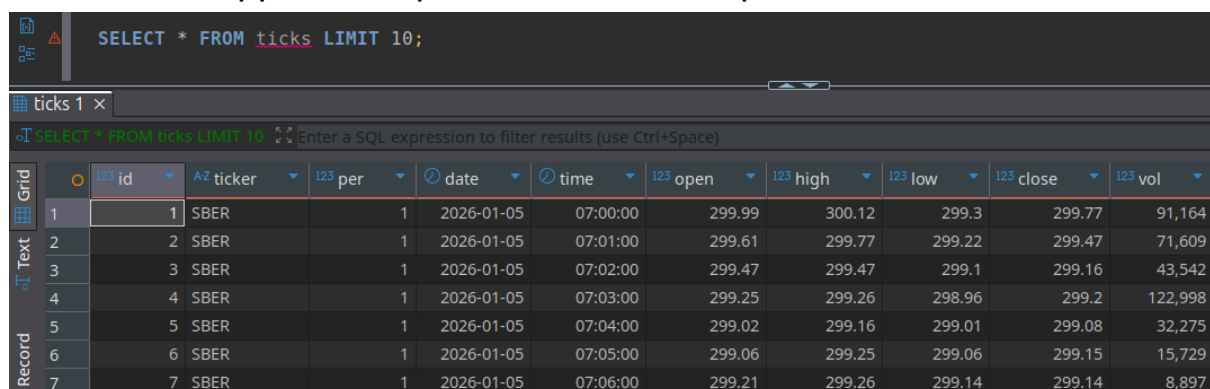


```
-- Проверка заполнения
SELECT ticker, COUNT(*) FROM ticks GROUP BY ticker;
```

	AZ ticker	count
1	LKOH	86,563
2	SBER	87,283

Скриншот 1.3. Проверка, что все данные вставлены успешно

Запрос и результат запроса проверки на то, что данные были вставлены корректно, представлены на скриншоте 1.4:



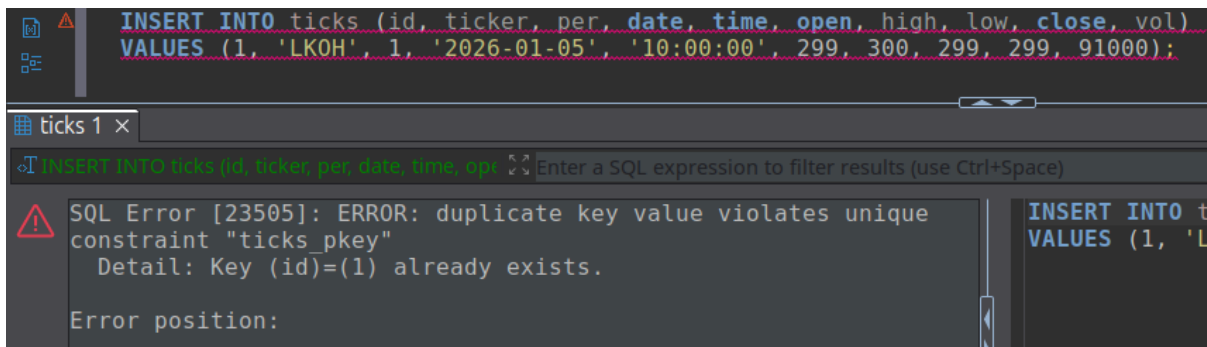
```
SELECT * FROM ticks LIMIT 10;
```

	id	AZ ticker	per	date	time	open	high	low	close	vol
1	1	SBER	1	2026-01-05	07:00:00	299.99	300.12	299.3	299.77	91,164
2	2	SBER	1	2026-01-05	07:01:00	299.61	299.77	299.22	299.47	71,609
3	3	SBER	1	2026-01-05	07:02:00	299.47	299.47	299.1	299.16	43,542
4	4	SBER	1	2026-01-05	07:03:00	299.25	299.26	298.96	299.2	122,998
5	5	SBER	1	2026-01-05	07:04:00	299.02	299.16	299.01	299.08	32,275
6	6	SBER	1	2026-01-05	07:05:00	299.06	299.25	299.06	299.15	15,729
7	7	SBER	1	2026-01-05	07:06:00	299.21	299.26	299.14	299.14	8,897

Скриншот 1.4. Проверка, что вставка выполнена корректно

Были выполнены CRUD-операции, нарушающие следующие созданные ограничения: PRIMARY KEY, NOT NULL, CHECK (high >= low), CHECK (per >= 0).

CRUD-операция, нарушающая ограничение PRIMARY KEY, и ошибка, всплывающая при ее выполнении, представлены на скриншоте 1.5:



```
INSERT INTO ticks (id, ticker, per, date, time, open, high, low, close, vol)
VALUES (1, 'LKOH', 1, '2026-01-05', '10:00:00', 299, 300, 299, 299, 91000);
```

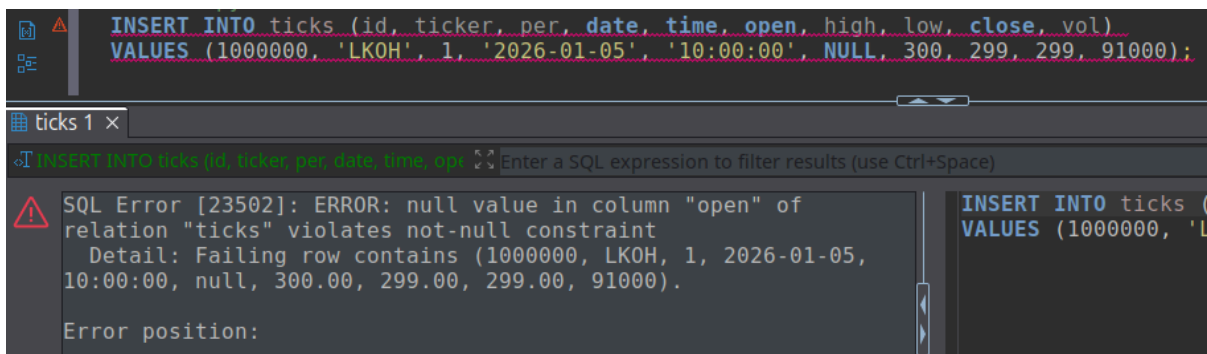
ticks 1 x

```
INSERT INTO ticks (id, ticker, per, date, time, open, high, low, close, vol)
VALUES (1, 'LKOH', 1, '2026-01-05', '10:00:00', 299, 300, 299, 299, 91000);
```

SQL Error [23505]: ERROR: duplicate key value violates unique constraint "ticks_pkey"
Detail: Key (id)=(1) already exists.
Error position:

Скриншот 1.5. Нарушение PRIMARY KEY

CRUD-операция, нарушающая ограничение NOT NULL, и ошибка, всплывающая при ее выполнении, представлены на скриншоте 1.6:



```
INSERT INTO ticks (id, ticker, per, date, time, open, high, low, close, vol)
VALUES (1000000, 'LKOH', 1, '2026-01-05', '10:00:00', NULL, 300, 299, 299, 91000);
```

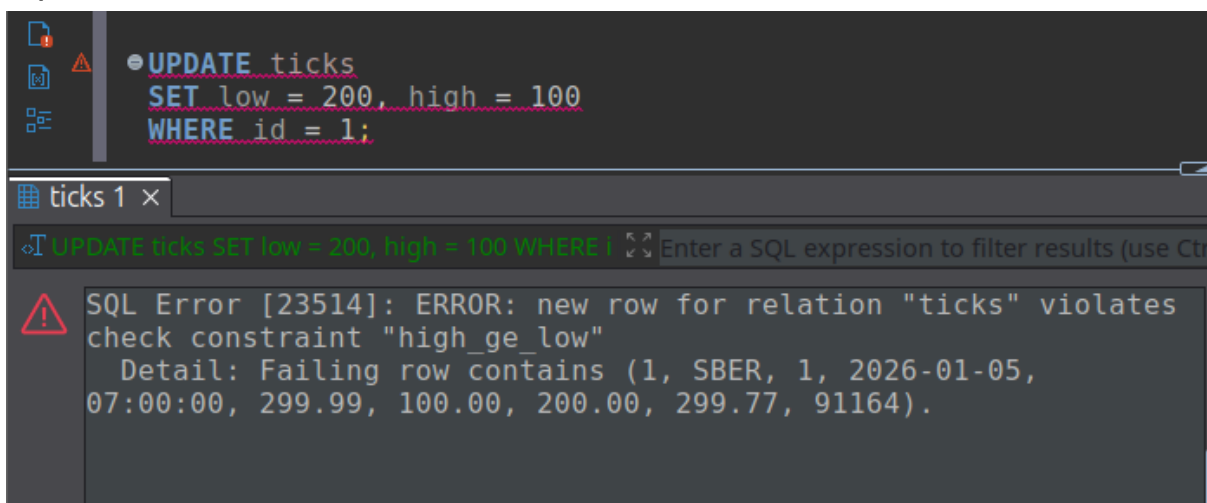
ticks 1 x

```
INSERT INTO ticks (id, ticker, per, date, time, open, high, low, close, vol)
VALUES (1000000, 'LKOH', 1, '2026-01-05', '10:00:00', NULL, 300, 299, 299, 91000);
```

SQL Error [23502]: ERROR: null value in column "open" of relation "ticks" violates not-null constraint
Detail: Failing row contains (1000000, LKOH, 1, 2026-01-05, 10:00:00, null, 300.00, 299.00, 299.00, 91000).
Error position:

Скриншот 1.6. Нарушение NOT NULL

CRUD-операция, нарушающая ограничение CHECK (high >= low), и ошибка, всплывающая при ее выполнении, представлены на скриншоте 1.7:



```
UPDATE ticks
SET low = 200, high = 100
WHERE id = 1;
```

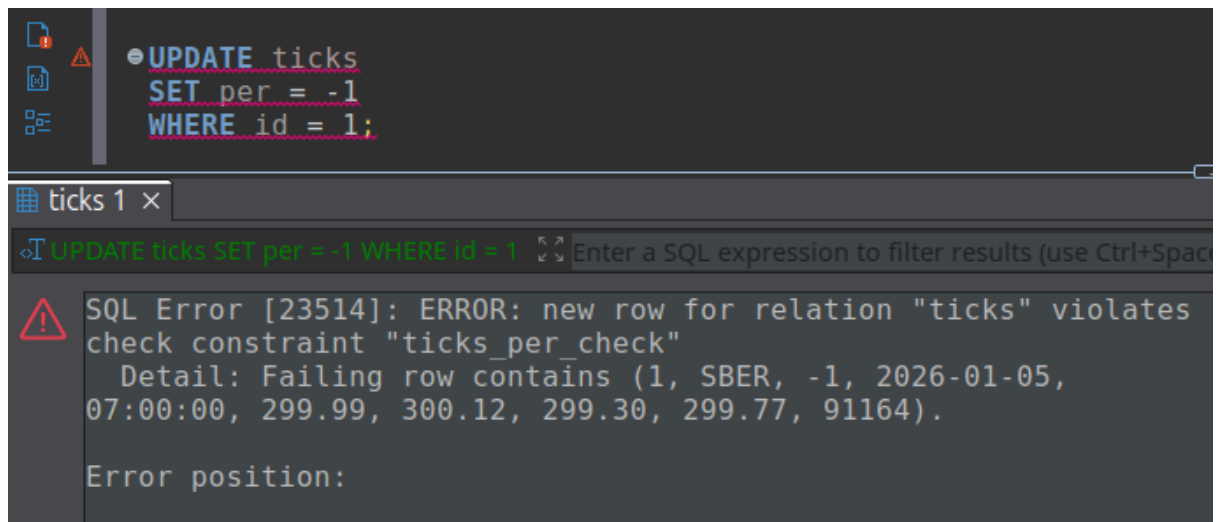
ticks 1 x

```
UPDATE ticks SET low = 200, high = 100 WHERE id = 1;
```

SQL Error [23514]: ERROR: new row for relation "ticks" violates check constraint "high_ge_low"
Detail: Failing row contains (1, SBER, 1, 2026-01-05, 07:00:00, 299.99, 100.00, 200.00, 299.77, 91164).

Скриншот 1.7. Нарушение CHECK (high >= low)

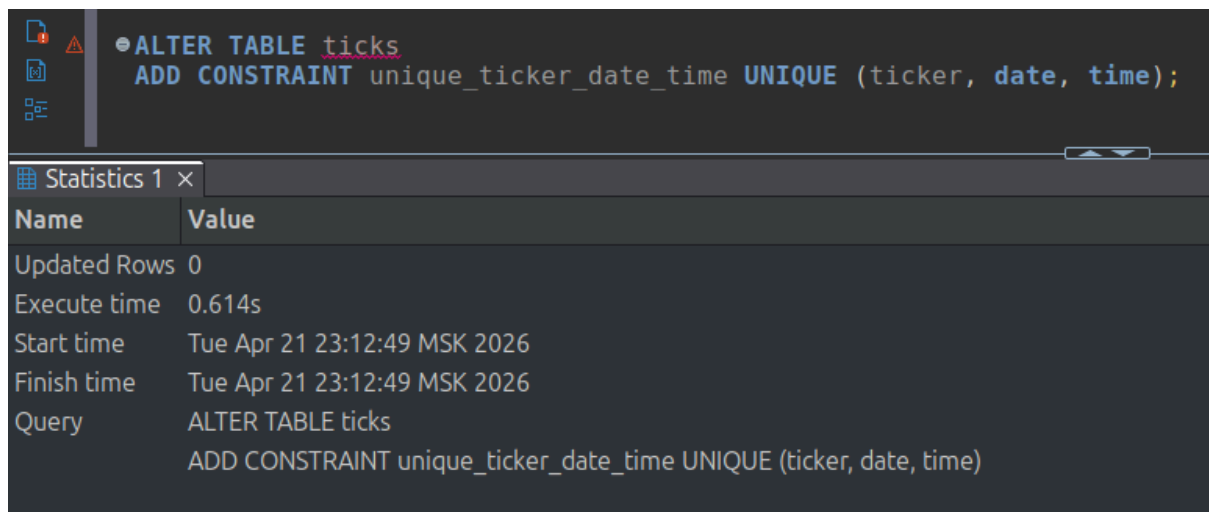
CRUD-операция, нарушающая ограничение CHECK ($per \geq 0$), и ошибка, всплывающая при ее выполнении, представлены на скриншоте 1.8:



Скриншот 1.8. Нарушение CHECK ($per \geq 0$)

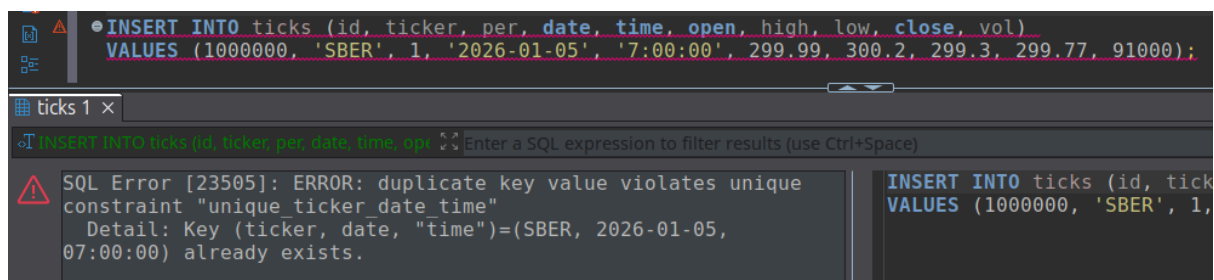
2. Добавить ограничение UNIQUE (пример: код бумаги дата). Попробовать выполнить CRUD-операции, нарушающие созданные ограничения.

Было добавлено ограничение UNIQUE на ticker, date, time. Запрос добавления представлен на скриншоте 2.1:



Скриншот 2.1. Добавление ограничения UNIQUE

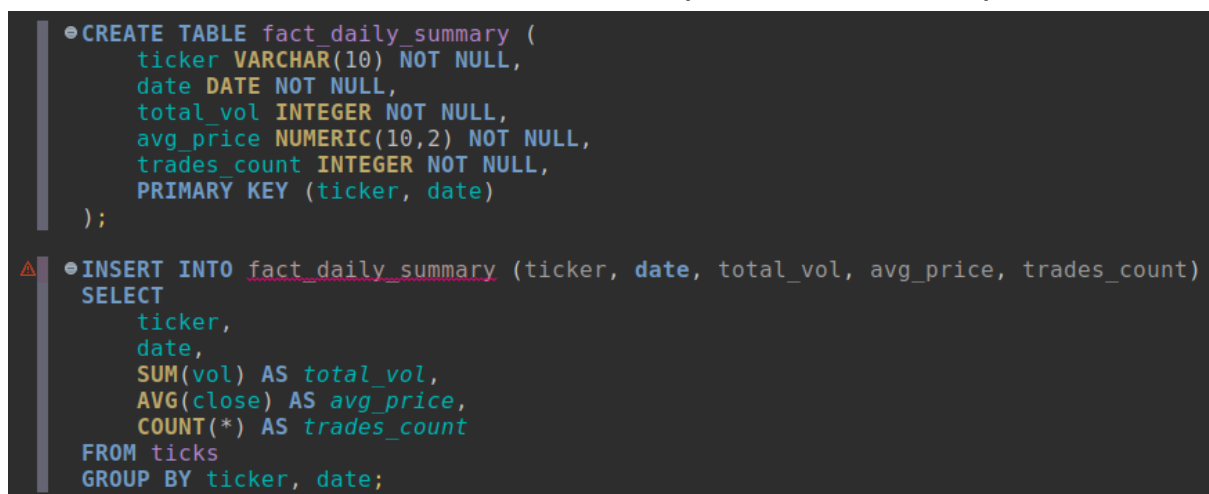
Попытка выполнить CRUD-операцию, нарушающее созданное ограничение представлена на скриншоте 2.2:



Скриншот 2.2. Нарушение ограничения UNIQUE

3. На основе данных из первой таблицы пункт 3 предыдущей темы (импортировать несколько csv по разным бумагам) построить две таблицы типа таблица фактов (котировки).

Сначала была построена таблица фактов «Дневные итоги», где первичный ключ состоит из полей ticker и date, что гарантирует уникальность каждой записи. В качестве мер были выбраны total_vol - суммарный объём торгов за день, avg_price - средняя цена закрытия 5-минутных свечей, и trades_count - количество 5-минутных интервалов с совершёнными сделками. Скрипт создания и заполнения этой таблицы представлен на скриншоте 3.1:



Скриншот 3.1. Создание и заполнение таблицы фактов «Дневные итоги»

Далее была построена таблица фактов «Максимумы/минимумы за день», где первичный ключ образован полями ticker и date. В качестве мер используются high - максимальная цена, достигнутая в течение торгового дня, low - минимальная цена за день, и total_vol - суммарный объём торгов. На таблицу наложено ограничение CHECK (high >= low),

гарантирующее логическую непротиворечивость данных. Скрипт создания и заполнения этой таблицы представлен на скриншоте 3.2:

```
CREATE TABLE fact_daily_range (  
    ticker VARCHAR(10) NOT NULL,  
    date DATE NOT NULL,  
    high NUMERIC(10,2) NOT NULL,  
    low NUMERIC(10,2) NOT NULL,  
    total_vol INTEGER NOT NULL,  
    PRIMARY KEY (ticker, date),  
    CONSTRAINT high_ge_low CHECK (high >= low)  
);  
  
INSERT INTO fact_daily_range (ticker, date, high, low, total_vol)  
SELECT  
    ticker,  
    date,  
    MAX(high) AS high,  
    MIN(low) AS low,  
    SUM(vol) AS total_vol  
FROM ticks  
GROUP BY ticker, date;
```

Скриншот 3.2. Создание и заполнение таблицы фактов
«Максимумы/минимумы за день»

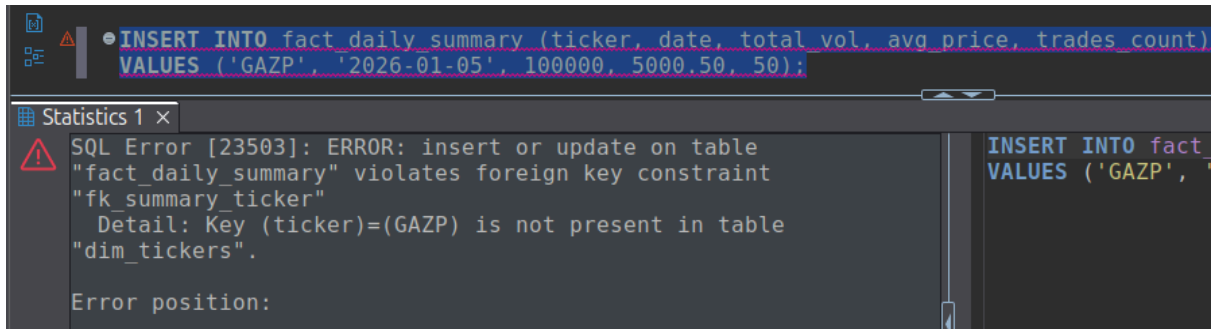
4. Построить справочник (ценные бумаги) с ограничением Foreign Key. Попытаться выполнить CRUD-операции, нарушающие созданные ограничения.

Скрипт создания, заполнения справочника ценных бумаг, а также накладывания ограничений FOREIGN KEY представлен на скриншоте 4.1:

```
CREATE TABLE dim_tickers (  
    ticker VARCHAR(10) PRIMARY KEY  
);  
  
INSERT INTO dim_tickers (ticker)  
SELECT DISTINCT ticker FROM ticks;  
  
ALTER TABLE fact_daily_summary  
ADD CONSTRAINT fk_summary_ticker  
FOREIGN KEY (ticker) REFERENCES dim_tickers(ticker);  
  
ALTER TABLE fact_daily_range  
ADD CONSTRAINT fk_range_ticker  
FOREIGN KEY (ticker) REFERENCES dim_tickers(ticker);
```

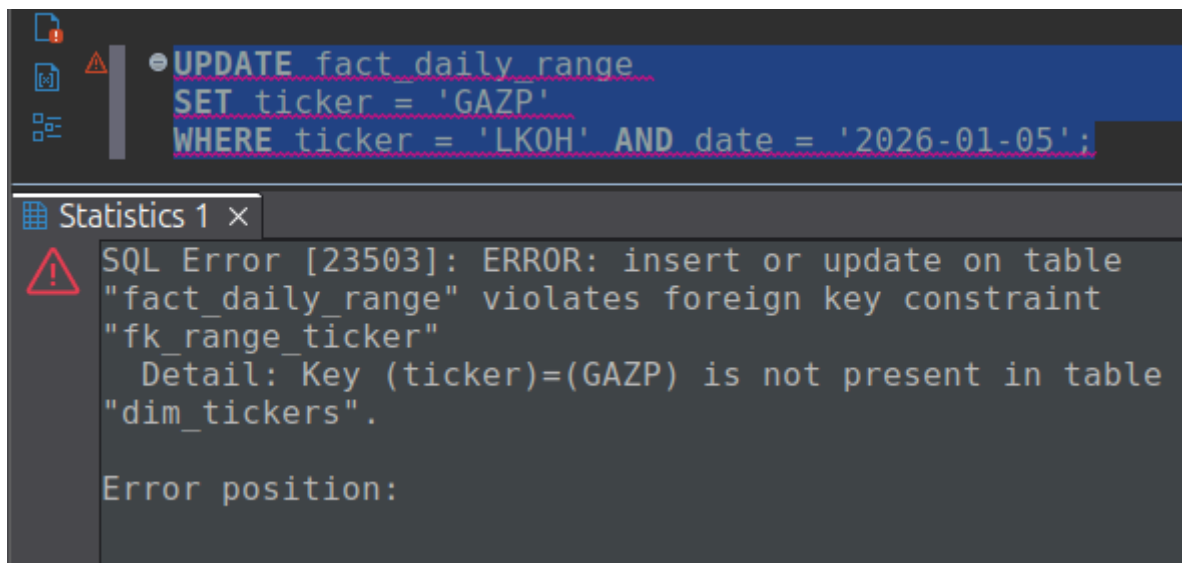
Скриншот 4.1. Создание, заполнение справочника ценных бумаг и наложение ограничений FOREIGN KEY

Попытка выполнить вставку в таблицу фактов «Дневные итоги», которая нарушает созданное ограничение, представлена на скриншоте 4.2:



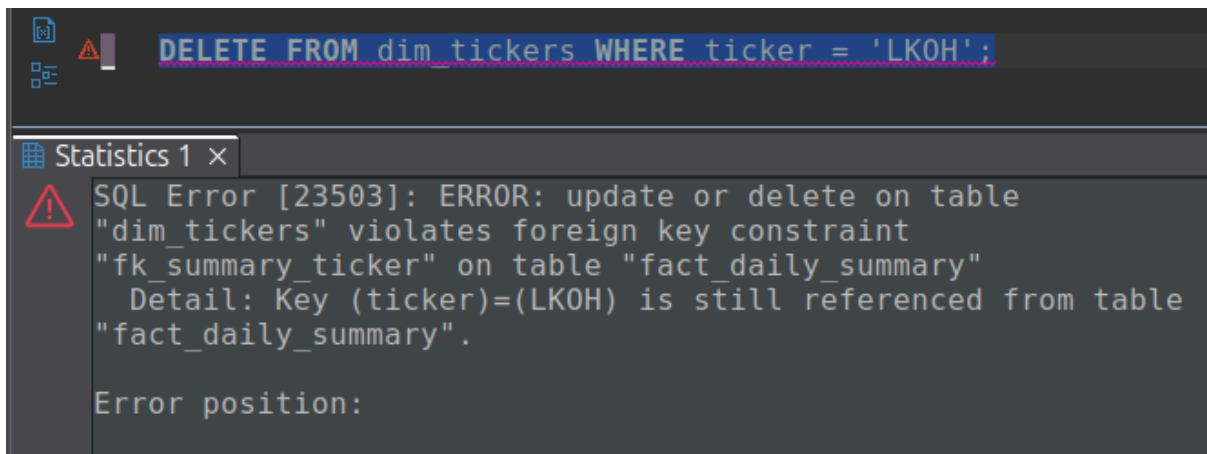
Скриншот 4.2. Вставка, нарушающая ограничение

Нарушающая ограничение попытка обновить запись таблицы фактов «Максимумы/минимумы за день» представлена на скриншоте 4.3:



Скриншот 4.3. Обновление, нарушающее ограничение

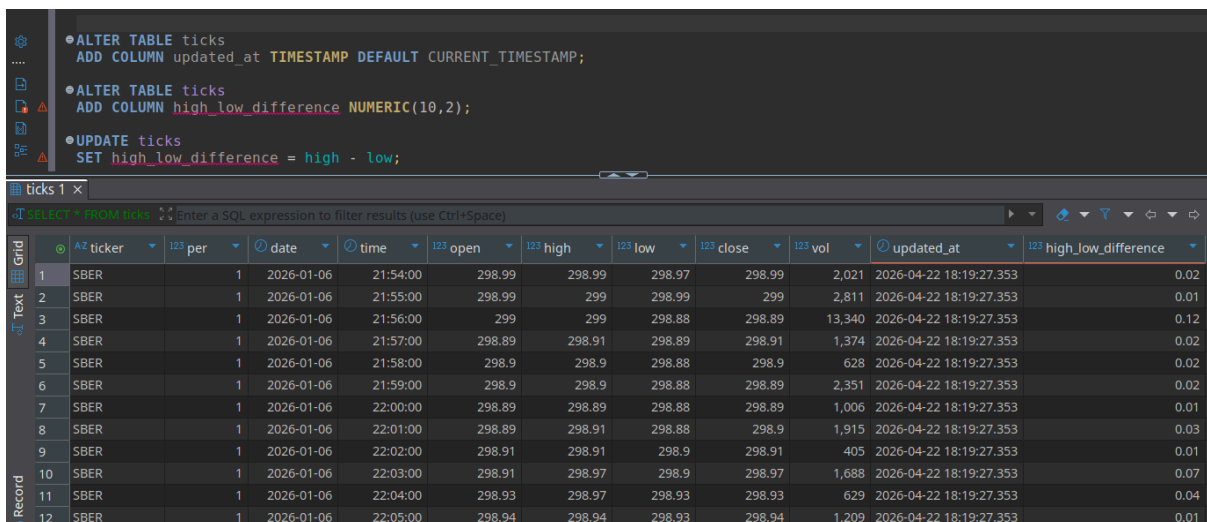
Нарушающая ограничение попытка удалить запись из справочника представлена на скриншоте 4.4:



Скриншот 4.4. Удаление, нарушающее ограничение

5. Выполнить несколько операций по добавлению/удалению/изменению типов полей таблицы п.1., включая поля, на которые наложены ограничения.

Были добавлены два новых поля: `updated_at` со значением по умолчанию `CURRENT_TIMESTAMP` и `high_low_difference`, которое потом заполняется как разность между `high` и `low` полями. После добавления `updated_at` сразу все ячейки были заполнены так как использовался `DEFAULT`, а при `high_low_difference` были пустыми, поэтому пришлось заполнять через `UPDATE`. Скрипт добавления этих двух полей представлен на скриншоте 5.1:



Скриншот 5.1. Добавление новых полей

Удалены новые поля, которые только что были добавлены, также был удален столбец `per`, на который были наложены ограничения `NOT NULL CHECK (per >= 0)`. Также я наложил

ограничение FOREIGN KEY на столбец ticker, а затем удалил этот столбец. Все удаления столбцов осуществились без проблем несмотря на наложенные на них ограничения.

```
① ALTER TABLE ticks DROP COLUMN updated_at;
① ALTER TABLE ticks DROP COLUMN high_low_difference;
① ALTER TABLE ticks DROP COLUMN per;

● ALTER TABLE ticks
  ADD CONSTRAINT fk_ticker
  FOREIGN KEY (ticker)
  REFERENCES dim_tickers(ticker);

ALTER TABLE ticks DROP COLUMN ticker;

SELECT * FROM ticks;
```

ticks 1 ×

SELECT * FROM ticks Enter a SQL expression to filter results (use Ctrl+Space)

	123 id	date	time	123 open	123 high	123 low	123 close	123 vol
1	1,875	2026-01-06	21:54:00	298.99	298.99	298.97	298.99	2,021
2	1,876	2026-01-06	21:55:00	298.99	299	298.99	299	2,811
3	1,877	2026-01-06	21:56:00	299	299	298.88	298.89	13,340
4	1,878	2026-01-06	21:57:00	298.89	298.91	298.89	298.91	1,374
5	1,879	2026-01-06	21:58:00	298.9	298.9	298.88	298.9	628
6	1,880	2026-01-06	21:59:00	298.9	298.9	298.88	298.89	2,351
7	1,881	2026-01-06	22:00:00	298.89	298.89	298.88	298.89	1,006

Скриншот 5.2. Удаление полей

Но стоит отметить, что если другие поля ссылаются на столбец, то удалить его будет нельзя. Такая ситуация представлена на скриншоте 5.3:

```
....
ALTER TABLE dim_tickers DROP COLUMN ticker;
```

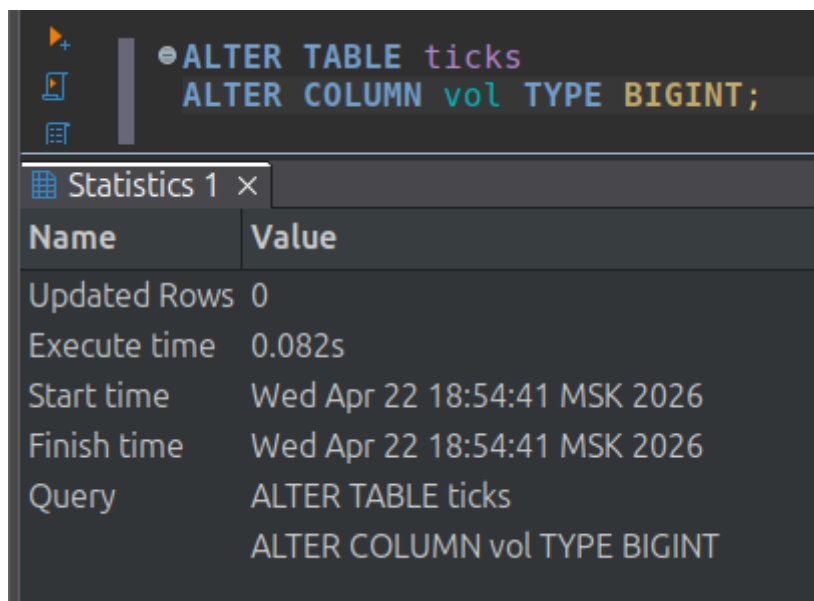
ticks 1 ×

ALTER TABLE dim_tickers DROP COLUMN ticker Enter a SQL expression to filter results (use Ctrl+Space)

SQL Error [2BP01]: ERROR: cannot drop column ticker of table dim_tickers because other objects depend on it
Detail: constraint fk_summary_ticker on table fact_daily_summary depends on column ticker of table dim_tickers
constraint fk_range_ticker on table fact_daily_range depends on column ticker of table dim_tickers
Hint: Use DROP ... CASCADE to drop the dependent objects too.

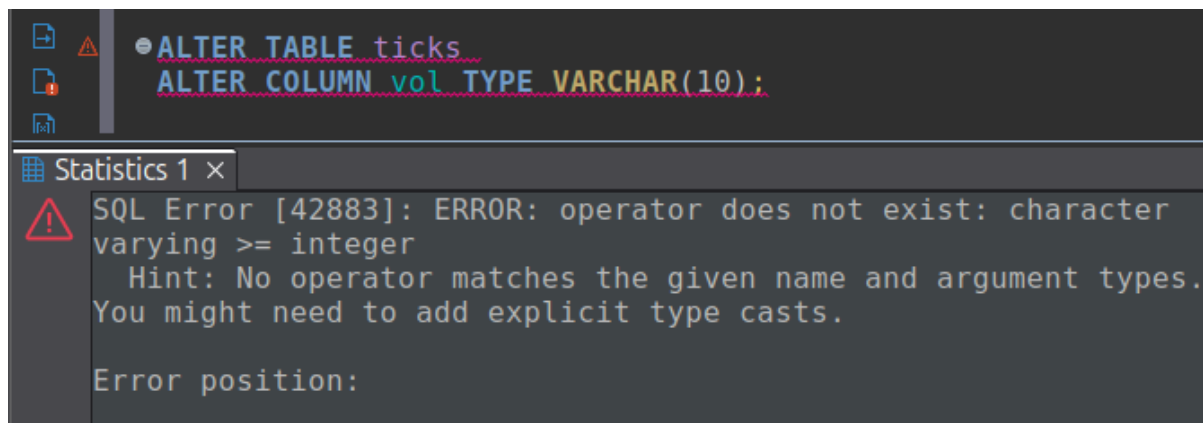
Скриншот 5.3. Удаление поля, на которое ссылаются другие поля

Изменение типа поля vol с INTEGER на BIGINT представлено на скриншоте 5.4:



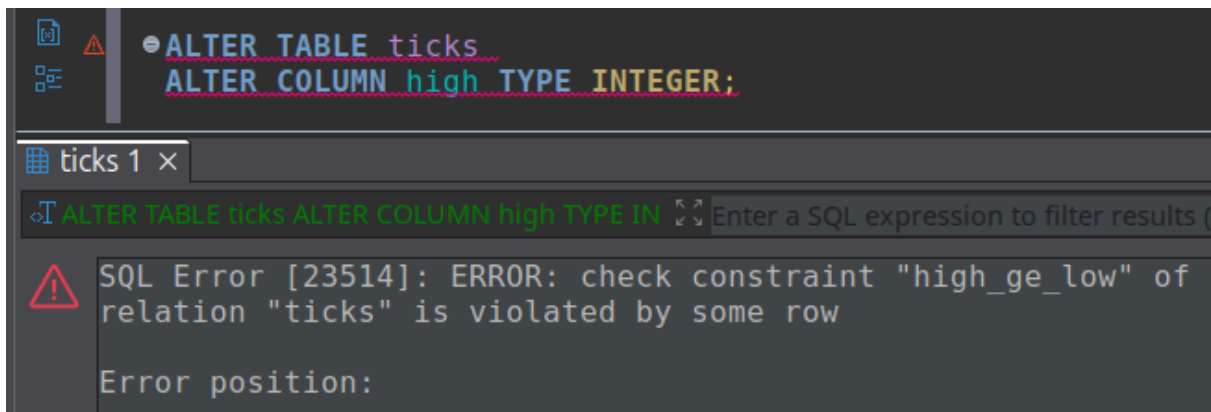
Скриншот 5.4. Изменение типа поля vol с INTEGER на BIGINT

Попытка изменить тип поля vol с BIGINT на VARCHAR(10) представлена на скриншоте 5.5. Произойдет ошибка, так как на это поле наложено ограничение CHECK (vol >= 0).



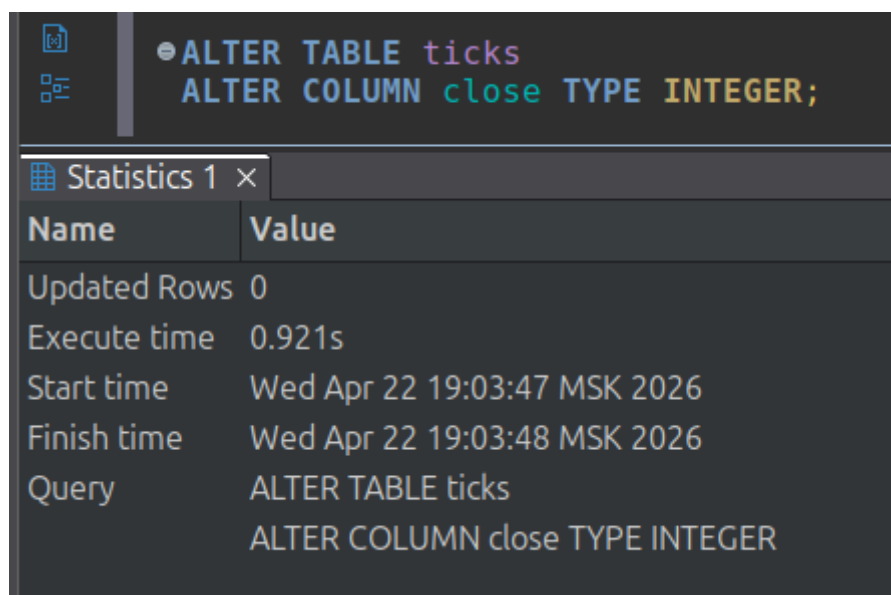
Скриншот 5.5. Изменение типа поля vol с BIGINT на VARCHAR(10)

Попытка изменить поля high с NUMERIC(10,2) на INTEGER проходит неуспешно из-за ограничения CONSTRAINT high_ge_low CHECK (high >= low):



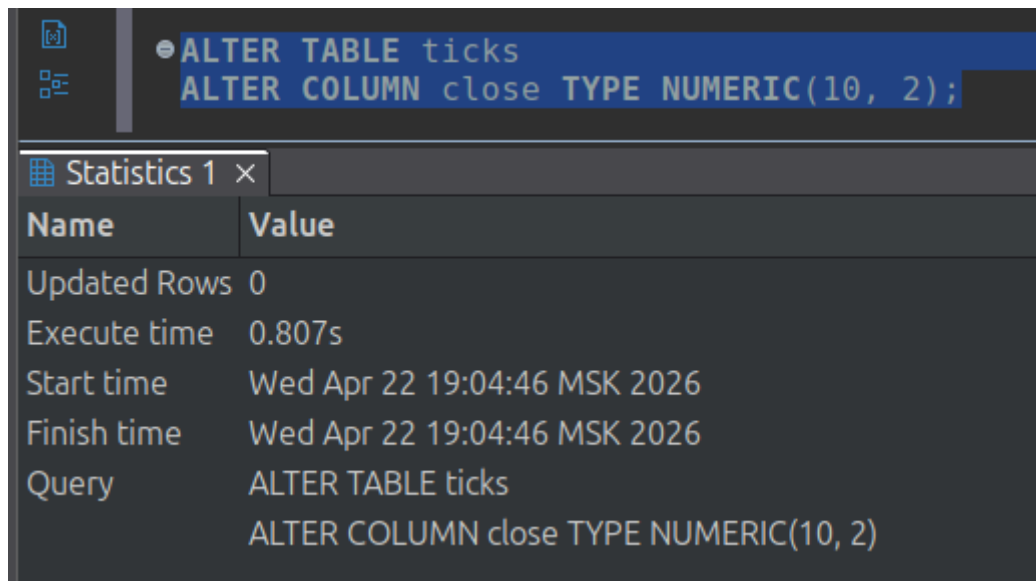
Скриншот 5.6. Попытка изменить тип поля high с NUMERIC(10, 2) на INTEGER

Но попытка изменить тип поля close с NUMERIC(10, 2) на INTEGER завершается успешно:



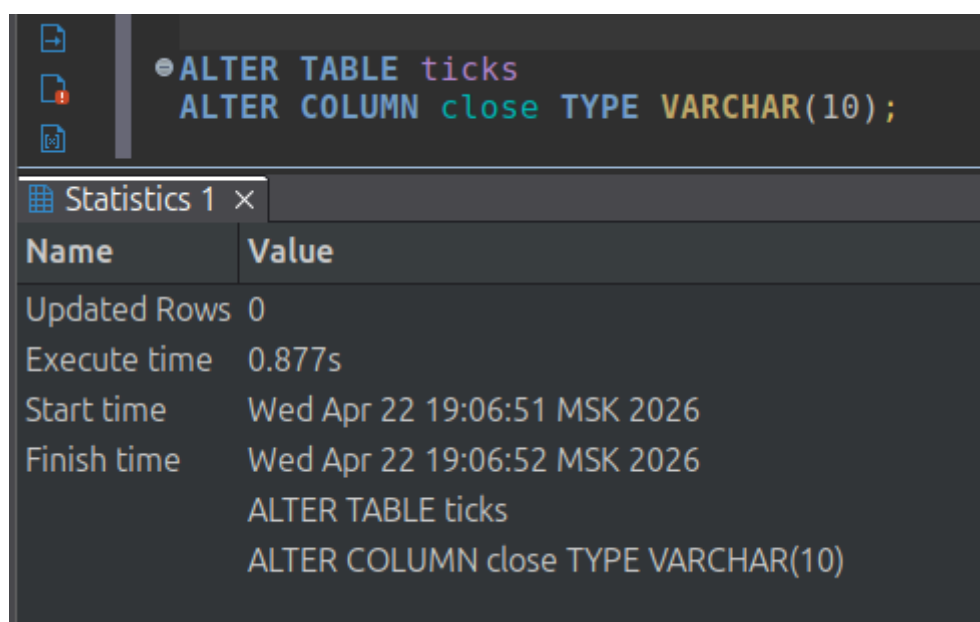
Скриншот 5.7. Попытка изменить тип поля close с NUMERIC(10, 2) на INTEGER

Обратно тоже можно с INTEGER на NUMERIC(10, 2):



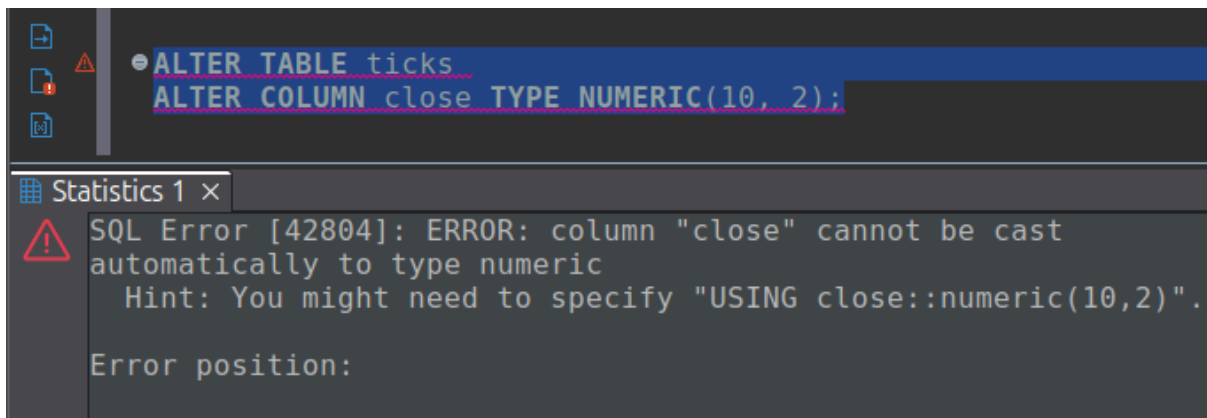
Скриншот 5.8. Попытка изменить тип поля close с INTEGER на NUMERIC(10, 2)

Изменить тип поля close с INTEGER/NUMERIC(10, 2) на VARCHAR(10) можно:



Скриншот 5.9. Попытка изменить тип поля close с NUMERIC(10, 2) на VARCHAR(10)

А чтобы преобразовать обратно возникают проблемы:



Скриншот 5.10. Попытка изменить тип поля close с VARCHAR(10) на INTEGER/NUMERIC(10, 2)

Стоит отметить замечание: промежуточное преобразование INTEGER в NUMERIC(10,2) изменило внутреннее представление ограничения, что позволило Postgres автоматически удалить его при финальном преобразовании NUMERIC(10,2) в VARCHAR. При прямом преобразовании INTEGER в VARCHAR система пыталась сохранить ограничение, обнаруживала несовместимость типов и блокировала операцию.

6. Удалить созданные ограничения (и потом создать заново).

Заново создадим таблицу ticks и заполним ее данными, чтобы могли работать с этой таблицей с чистого листа после экспериментов в прошлом пункте.

Таблица ticks содержит следующие ограничения:

The screenshot shows a database client interface. At the top, a SQL query is entered in a text editor:

```
SELECT conname AS constraint_name,  
       contype AS constraint_type,  
       pg_get_constraintdef(oid) AS definition  
FROM pg_constraint  
WHERE conrelid = 'ticks'::regclass  
ORDER BY contype, conname;
```

Below the query editor, a tab labeled "pg_constraint 1" is active. A filter bar shows the query: "SELECT conname AS constraint_name, contype". Below this, a table displays the results of the query. The table has three columns: "constraint_name", "constraint_type", and "definition". The results are as follows:

	constraint_name	constraint_type	definition
1	high_ge_low	c	CHECK ((high >= low))
2	ticks_close_check	c	CHECK ((close > (0)::numeric))
3	ticks_high_check	c	CHECK ((high > (0)::numeric))
4	ticks_low_check	c	CHECK ((low > (0)::numeric))
5	ticks_open_check	c	CHECK ((open > (0)::numeric))
6	ticks_per_check	c	CHECK ((per >= 0))
7	ticks_vol_check	c	CHECK ((vol >= 0))
8	ticks_pkey	p	PRIMARY KEY (id)

Скриншот 6.1. Вывод всех ограничений таблицы ticks

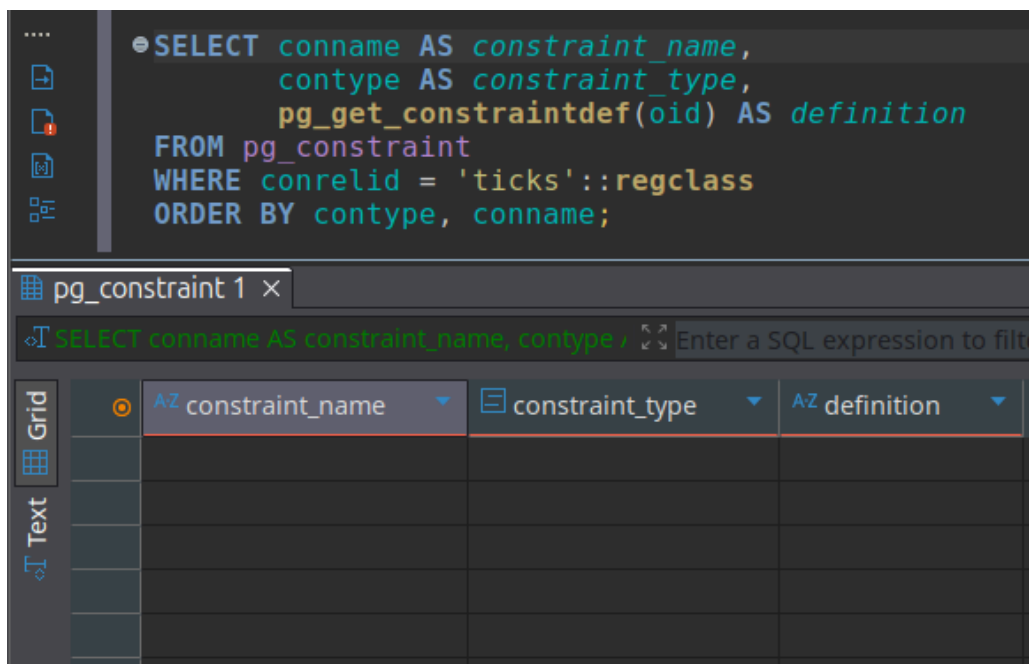
Удалим их все, а также удалим ограничения NOT NULL:

ALTER TABLE ticks DROP CONSTRAINT ticks_pkey;
ALTER TABLE ticks DROP CONSTRAINT ticks_vol_check;
ALTER TABLE ticks DROP CONSTRAINT ticks_per_check;
ALTER TABLE ticks DROP CONSTRAINT ticks_open_check;
ALTER TABLE ticks DROP CONSTRAINT ticks_low_check;
ALTER TABLE ticks DROP CONSTRAINT ticks_high_check;
ALTER TABLE ticks DROP CONSTRAINT ticks_close_check;
ALTER TABLE ticks DROP CONSTRAINT high_ge_low;
ALTER TABLE ticks ALTER COLUMN ticker DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN per DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN date DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN time DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN open DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN high DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN low DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN close DROP NOT NULL;
ALTER TABLE ticks ALTER COLUMN vol DROP NOT NULL;

Statistics 1 x	
Name	Value
Updated Rows	0
Execute time	0.034s
Start time	Wed Apr 22 19:27:00 MSK 2026
Finish time	Wed Apr 22 19:27:00 MSK 2026
Query	ALTER TABLE ticks DROP CONSTRAINT ticks_pkey; ALTER TABLE ticks DROP CONSTRAINT ticks_vol_check; ALTER TABLE ticks DROP CONSTRAINT ticks_per_check; ALTER TABLE ticks DROP CONSTRAINT ticks_open_check; ALTER TABLE ticks DROP CONSTRAINT ticks_low_check; ALTER TABLE ticks DROP CONSTRAINT ticks_high_check; ALTER TABLE ticks DROP CONSTRAINT ticks_close_check; ALTER TABLE ticks DROP CONSTRAINT high_ge_low; ALTER TABLE ticks ALTER COLUMN ticker DROP NOT NULL; ALTER TABLE ticks ALTER COLUMN per DROP NOT NULL; ALTER TABLE ticks ALTER COLUMN date DROP NOT NULL; ALTER TABLE ticks ALTER COLUMN time DROP NOT NULL; ALTER TABLE ticks ALTER COLUMN open DROP NOT NULL; ALTER TABLE ticks ALTER COLUMN high DROP NOT NULL;

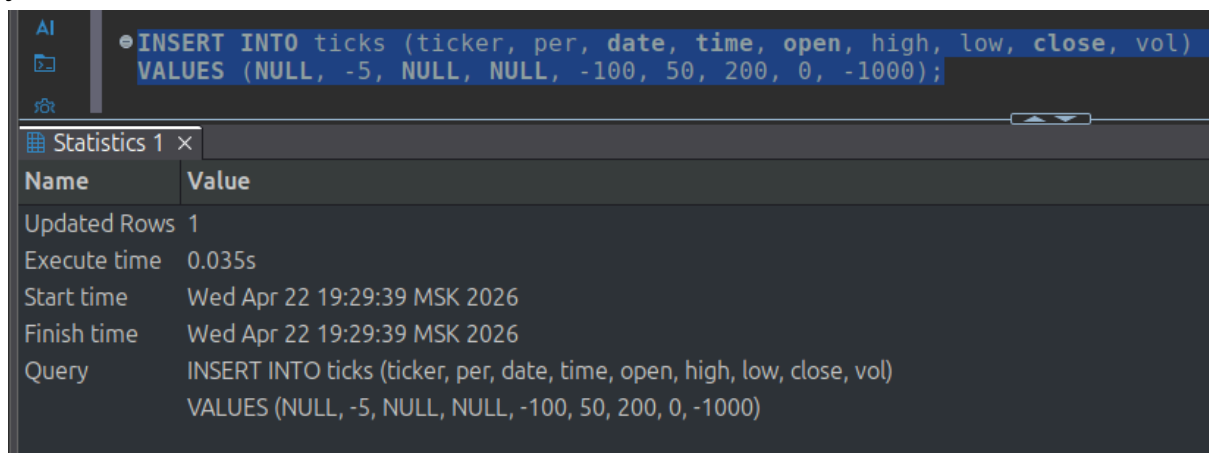
Скриншот 6.2. Удаление всех ограничений таблицы ticks

Теперь видим, что вывод ограничений пустой:



Скриншот 6.3. Вывод всех ограничений таблицы ticks после удаления

Попробуем вставить NULL и увидим, что вставка проходит успешно:



Скриншот 6.4. Попытка вставки

Теперь вернем все ограничения, которые были удалены. Для начала установим всем столбцам ограничение NOT NULL:

```
ALTER TABLE ticks ALTER COLUMN ticker SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN per SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN date SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN time SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN open SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN high SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN low SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN cclose SET NOT NULL;  
ALTER TABLE ticks ALTER COLUMN vol SET NOT NULL;
```

Скриншот 6.5. Возвращение NOT NULL ограничений

Далее вернем PRIMARY KEY и CHECK ограничения:

```
ALTER TABLE ticks ADD PRIMARY KEY (id);  
ALTER TABLE ticks ADD CONSTRAINT ticks_per_check CHECK (per >= 0);  
ALTER TABLE ticks ADD CONSTRAINT ticks_open_check CHECK (open > 0);  
ALTER TABLE ticks ADD CONSTRAINT ticks_high_check CHECK (high > 0);  
ALTER TABLE ticks ADD CONSTRAINT ticks_low_check CHECK (low > 0);  
ALTER TABLE ticks ADD CONSTRAINT ticks_cclose_check CHECK (cclose > 0);  
ALTER TABLE ticks ADD CONSTRAINT ticks_vol_check CHECK (vol >= 0);  
ALTER TABLE ticks ADD CONSTRAINT high_ge_low CHECK (high >= low);
```

Скриншот 6.6. Возвращение PRIMARY KEY и CHECK ограничений

Делаем проверку, что все вернулось на свои места:

```

SELECT conname AS constraint_name,
       contype AS constraint_type,
       pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE conrelid = 'ticks'::regclass
ORDER BY contype, conname;

```

	constraint_name	constraint_type	definition
1	high_ge_low	c	CHECK ((high >= low))
2	ticks_close_check	c	CHECK ((close > (0)::numeric))
3	ticks_high_check	c	CHECK ((high > (0)::numeric))
4	ticks_low_check	c	CHECK ((low > (0)::numeric))
5	ticks_open_check	c	CHECK ((open > (0)::numeric))
6	ticks_per_check	c	CHECK ((per >= 0))
7	ticks_vol_check	c	CHECK ((vol >= 0))
8	ticks_pkey	p	PRIMARY KEY (id)

Скриншот 6.7. Вывод всех ограничений таблицы ticks после возвращения

Вставка NULL тоже блокируется:

```

INSERT INTO ticks (ticker, per, date, time, open, high, low, close, vol)
VALUES (NULL, -5, NULL, NULL, -100, 50, 200, 0, -1000);

```

constraint_name	constraint_type	definition
high_ge_low	c	CHECK ((high >= low))
ticks_close_check	c	CHECK ((close > (0)::numeric))
ticks_high_check	c	CHECK ((high > (0)::numeric))
ticks_low_check	c	CHECK ((low > (0)::numeric))
ticks_open_check	c	CHECK ((open > (0)::numeric))
ticks_per_check	c	CHECK ((per >= 0))
ticks_vol_check	c	CHECK ((vol >= 0))
ticks_pkey	p	PRIMARY KEY (id)

SQL Error [23502]: ERROR: null value in column "ticker" of relation "ticks" violates not-null constraint
 Detail: Failing row contains (173848, null, -5, null, null, -100.00, 50.00, 200.00, 0.00, -1000).

Скриншот 6.8. Попытка вставить строку с NULL

7. Добавить несколько индексов на таблицу разных типов. Добавить индексы, для быстрого получения результата по запросу бумага, дата котировки.

Были созданы следующие индексы:

```
CREATE INDEX idx_ticks_ticker_date ON ticks (ticker, date);
CREATE INDEX idx_ticks_ticker_hash ON ticks USING HASH (ticker);
CREATE INDEX idx_ticks_lkoh_date ON ticks (date) WHERE ticker = 'LKOH';
```

Скриншот 7.1. Создание индексов

Результат благодаря индексному сканированию получаем быстрее:

EXPLAIN ANALYZE SELECT * FROM ticks WHERE ticker = 'SBER' AND date = '2026-01-05';	
1	Bitmap Heap Scan on ticks (cost=14.17..1675.49 rows=963 width=53) (actual time=0.123..2.271 rows=995 loops=1)
2	Recheck Cond: (((ticker)::text = 'SBER'::text) AND (date = '2026-01-05'::date))
3	Heap Blocks: exact=12
4	-> Bitmap Index Scan on idx_ticks_ticker_date (cost=0.00..13.93 rows=963 width=0) (actual time=0.094..0.096 rows=995 loops=1)
5	Index Cond: (((ticker)::text = 'SBER'::text) AND (date = '2026-01-05'::date))
6	Planning Time: 0.208 ms
7	Execution Time: 4.385 ms

Скриншот 7.2. Выполнение запроса с индексом

Без индекса тот же запрос выполняется вот так (медленнее):

EXPLAIN ANALYZE SELECT * FROM ticks WHERE ticker = 'SBER' AND date = '2026-01-05';	
1	Seq Scan on ticks (cost=0.00..4591.69 rows=963 width=53) (actual time=0.024..19.168 rows=995 loops=1)
2	Filter: (((ticker)::text = 'SBER'::text) AND (date = '2026-01-05'::date))
3	Rows Removed by Filter: 172851
4	Planning Time: 0.129 ms
5	Execution Time: 21.181 ms

Скриншот 7.3. Выполнение запроса без индекса

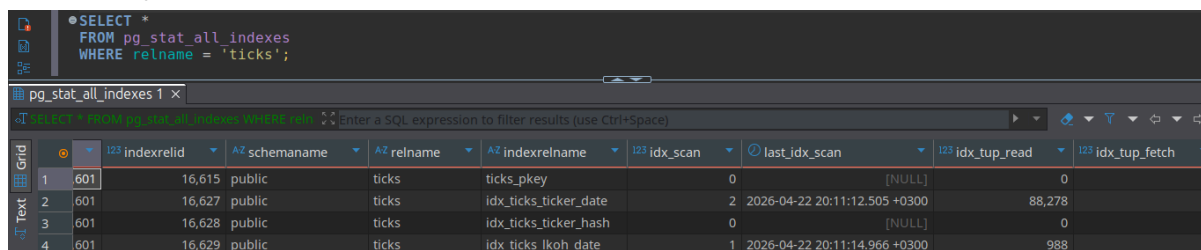
Запрос с индексом выполняется быстрее. На больших данных разница была бы более существенной.

8. Посмотреть статистические сведения об индексах.

pg_indexes - доступ к полезной информации обо всех индексах в базе данных.

pg_stat_all_indexes — это системное представление Postgres, которое содержит статистику использования всех индексов в текущей базе данных

В pg_stat_all_indexes мы можем отслеживать устаревшие индексы. Там видно как часто и насколько эффективно используются индексы:

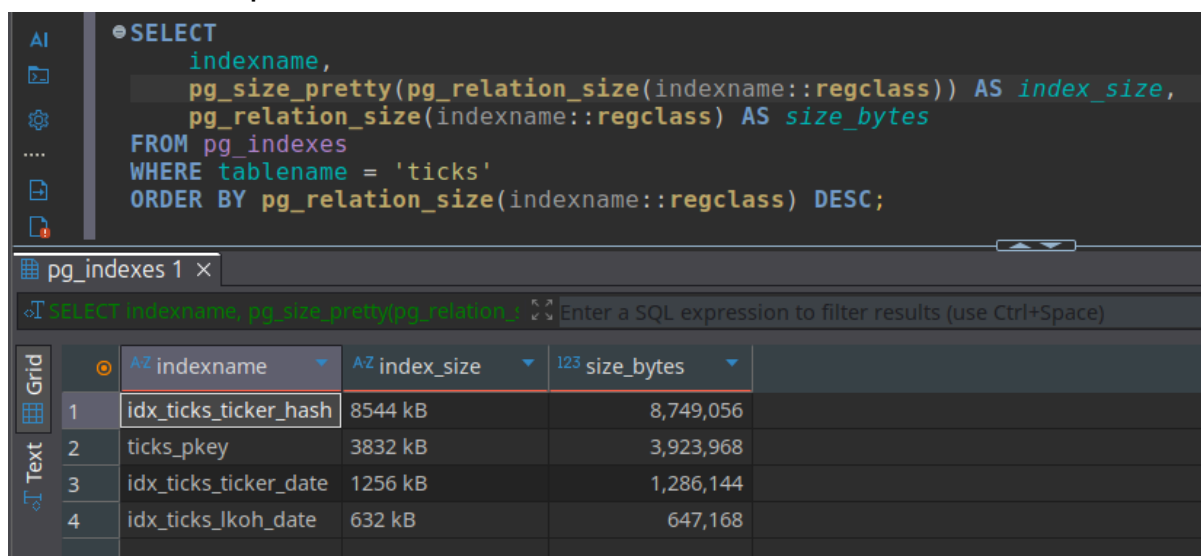


```
SELECT *
FROM pg_stat_all_indexes
WHERE relname = 'ticks';
```

	i23 indexrelid	A2 schemaname	A2 relname	A2 indexrelname	i23 idx_scan	last_idx_scan	i23 idx_tup_read	i23 idx_tup_fetch
1	601	16,615	public	ticks	ticks_pkey	0	[NULL]	0
2	601	16,627	public	ticks	idx_ticks_ticker_date	2	2026-04-22 20:11:12.505 +0300	88,278
3	601	16,628	public	ticks	idx_ticks_ticker_hash	0	[NULL]	0
4	601	16,629	public	ticks	idx_ticks_lkoh_date	1	2026-04-22 20:11:14.966 +0300	988

Скриншот 8.1. Содержимое pg_stat_all_indexes по таблице ticks

А с помощью pg_indexes мы можем посмотреть пространство на диске, которое занимают индексы:



```
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size,
    pg_relation_size(indexname::regclass) AS size_bytes
FROM pg_indexes
WHERE tablename = 'ticks'
ORDER BY pg_relation_size(indexname::regclass) DESC;
```

	A2 indexname	A2 index_size	i23 size_bytes
1	idx_ticks_ticker_hash	8544 kB	8,749,056
2	ticks_pkey	3832 kB	3,923,968
3	idx_ticks_ticker_date	1256 kB	1,286,144
4	idx_ticks_lkoh_date	632 kB	647,168

Скриншот 8.2. Размер индексов

Стоит отметить, что индекса 4 вместо 3 созданных. Это потому что для PRIMARY KEY полей индекс создается автоматически.

9. На основе таблицы создать материализованное представление (например, с группировкой по объему торгов ежемесячно).

Создадим такое материализованное представление на основе таблицы ticks:

The screenshot displays a database IDE interface. The top pane shows a SQL query to create a materialized view named 'monthly_ticks'. The query selects various fields from the 'ticks' table, grouped by ticker, year, and month. The bottom pane shows the execution statistics for this query.

```
CREATE MATERIALIZED VIEW monthly_ticks AS
SELECT
    ticker,
    EXTRACT(YEAR FROM date) AS year,
    EXTRACT(MONTH FROM date) AS month,
    COUNT(*) AS bars_count,
    SUM(vol) AS total_volume,
    AVG(close) AS avg_close,
    MIN(low) AS month_low,
    MAX(high) AS month_high
FROM ticks
GROUP BY ticker, EXTRACT(YEAR FROM date), EXTRACT(MONTH FROM date)
ORDER BY ticker, year, month;
```

Name	Value
Updated Rows	8
Execute time	0.322s
Start time	Wed Apr 22 20:17:38 MSK 2026
Finish time	Wed Apr 22 20:17:38 MSK 2026
Query	CREATE MATERIALIZED VIEW monthly_ticks AS SELECT ticker, EXTRACT(YEAR FROM date) AS year, EXTRACT(MONTH FROM date) AS month, COUNT(*) AS bars_count, SUM(vol) AS total_volume, AVG(close) AS avg_close, MIN(low) AS month_low, MAX(high) AS month_high FROM ticks GROUP BY ticker, EXTRACT(YEAR FROM date), EXTRACT(MONTH FROM date) ORDER BY ticker, year, month

Скриншот 9.1. Создание материализованного представления